BESCHREIBUNG

Assemblerdokumentation

Cross-Assembler c82ass: Installation, Sprachdefinition, Anweisungen, Makros und Compilierungsbeispiel

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205_Assembler_Dokumentation.md	1.00-039
Stand	25.10.2026	
Umfang	21 Seiten, A4 hoch	
Erzeugt mit	erzeuge_assemblerdokumentation_pdf.py	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Inhaltsverzeichnis

Cross-Assembler für C8205 / C8205Z – Version 1.00-039 . . . 3

1. Installation . . . 3
1.1 Verzeichnisstruktur . . . 3

2. Überblick . . . 4

3. Der Assembler c82ass . . . 6
3.1 Programmstart . . . 6
3.1a Schalter --ABSPR (absolut adressierter KEP-Lochstreifen) . . . 8
3.1b Vorspann der .test-Datei (Testmarken im Quelltext) . . . 9
3.1c Nachspann der .test-Datei (Prüfungs-Testmarken im Quelltext) . . . 10
3.2 Sprachdefinition . . . 11
3.3 Assembleranweisungen . . . 12
3.3.1 Definiere Bereich . . . 12
3.3.2 Definiere Bereich auf Null . . . 12
3.3.3 Definiere Wert . . . 12
3.3.4 Definiere Adresse . . . 12
3.3.5 Definiere Text zur Ausgabe von Zeichen . . . 12
3.3.6 Definiere Zeichen (spezielle Sonderzeichen) . . . 13
3.3.7 Beginn des Quellcodes . . . 13
3.3.8 Definition Relativadresse . . . 14
3.3.9 Definition Absolutadresse . . . 14
3.3.10 Definition Absolute Sektoradresse . . . 14
3.3.11 Ende des verschieblichen Teils des Quelltextes . . . 14
3.3.12 Pragma LEITADRESSE . . . 14
Tabelle 2: Sonderzeichen für die dz-Anweisung . . . 16
Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz . . . 16

4. Der Makro-Präprozessor des Assemblers . . . 17
4.1 Makros im Quelltext . . . 19

5. Compilierungsbeispiel . . . 20
5.1 Quelltext bsp_5_16.asm und Assemblerlauf . . . 20
5.2 Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für die Eingabe mit Kennzeichen (Format des Minimalprogramms) . . . 21

Version: 1.00-039

Diese Datei ist eine Textableitung von `C8205_Assembler_Dokumentation.odt` in der Version 1.00-039, gedacht zum Durchsuchen und Zitieren. **Maßgeblich bleibt die .odt** (siehe `C8205-Projektstruktur.md`, Abschnitte 1 und 3); bei Widersprüchen gilt die .odt. Diese Ableitung wird nicht automatisch aktualisiert, wenn die .odt sich ändert.

Stand 25.10.2026 (1.00-039): Die .odt trägt jetzt 1.00-039; die Usage-Ausgabe, `--version` und das Beispiel 5.1 nennen die Programmversion 1.00-038 (Programm: Wiederholungsbefehle, Startsektor für n Wiederholungen jetzt Tabelle[32 − n]; `!$wdhl_adr_e` verlangt als zweiten Operanden den Anfang der Nullspur, die das Programm selbst bereitstellt).

Stand 25.10.2026 (1.00-038): Die .odt trägt jetzt ebenfalls 1.00-038 (Programm c82ass bleibt 1.00-037). Berichtigt in .odt und .md: Verweise auf `c82load`, `c82dump` und `c82simi` entfernt (1.1, 3.1, 3.3.7), Nullwort `dw 0` als `0.` (3.3.3), Leitbefehl `<start> 240:` mit Doppelpunkt (3.1a), Versionszeile im Beispiel (5.1), „Objektdatei“ in 1.1. Außerdem die .md an die .odt angeglichen: Zusätze entfernt, die nur die .md hatte (Dateiliste 3.1: `.obj` und `.dmp`; `dw` in 3.3.3; Anmerkung zur Zeile „-“ der Tabelle 2), fehlender Satz zur Lochbandleser-Ablage in 3.1 ergänzt, Anordnung des Absatzes zu `asm_alle.sh` in 1.1 wie in der .odt; `.odt`, Programm und Beispielausgaben unverändert (1.00-037). Zuvor, Stand 24.10.2026 (1.00-037): Die Objektdatei `.obj` hat kein Zeilenende mehr (nur Wörter mit `.` und am Ende `/`); davor (1.00-036) ist sie jetzt ein binär codierter R300-Lochstreifen (je Zeichen eine Zeile aus acht Ziffern 0/1, ohne Zeilenende-Zeichen; ein anderes Format gibt es nicht mehr; Abschnitt 3 und 5.2). Zuvor, Stand 24.10.2026 (1.00-035): Die Objektdatei `.obj` enthält keine Leerzeichen mehr (Abschnitt 3 und 5.2; Beispiele der Objektzeilen entsprechend geändert); Programm, Titelzeile, Usage- und `--version`-Ausgabe 1.00-037. Zuvor, Stand 24.10.2026 (1.00-034): Textgestalt bereinigt (Leerzeile vor drei Listen, damit das Layoutskript keinen Hinweis mehr meldet); Inhalt und Abschnitte unverändert, die Titelzeile der .odt trägt jetzt ebenfalls 1.00-034. Zuvor, Stand 23.10.2026 (1.00-033): Abschnitt 2 beschreibt die Fehlerobergrenze (höchstens 20 Fehlerzeilen) und die Zusammenfassung gleicher Fehlermeldungen (ab Assembler 1.00-034); Titelzeile 1.00-033, Beispielausgaben des Programms 1.00-034. Zuvor (1.00-032): Abschnitt 3.1c beschreibt den Nachspann der `.test`-Datei genau (Abschnitt `Prüfungen:`, Semikolon entfällt, Beispiel); der Assembler setzt das um. Die Ableitung ist neu aus der geänderten .odt erstellt und gibt deren Abschnitte 1 bis 5 wieder. Die Versionsnummer der .odt wird bei jeder Änderung erhöht (jetzt 1.00-037: Titelzeile; die Usage-Ausgabe und die Beschreibung von `--version` nennen die Programmversion 1.00-037); die .md zählt ihren Build selbst (jetzt 1.00-038) und nennt im Kopf die Nummer der .odt, von der sie abgeleitet ist. Einen Abschnitt 6 gibt es weder in der .odt noch in dieser Ableitung. Offensichtliche Tippfehler der .odt sind bereinigt; Überschriften, Codeblöcke und Tabellen haben die Form der Markdown-Ableitung. Das PDF `C8205-Assemblerdokumentation_A4.pdf` ist aus dieser Fassung erzeugt.

Cross-Assembler für C8205 / C8205Z – Version 1.00-039

Die gesamte Software läuft sowohl unter Linux als auch Windows 10. Sie ist in C (gcc) programmiert.

1. Installation

Zum Zweck der Installation ist die Datei `C8205.zip` in einem geeigneten Verzeichnis zu entpacken. Beim Entpacken wird ein Unterverzeichnis angelegt, in dem sich sowohl Windows- als auch Linux-Binaries befinden.

1.1 Verzeichnisstruktur

Der Assembler liegt im Unterverzeichnis `Assembler` des Gesamtprojekts C8205. Es enthält folgende Struktur (Beispielprojekte gekürzt):

```
make_c82ass.sh                # Skript zum Erzeugen des Assemblers unter Linux
c82ass.sh                     # Ausführen des Assemblers
asm_alle.sh                   # assembliert alle .asm-Dateien unterhalb eines Verzeichnisses (meldet nur Fehler)
                                asm_alle.sh [Basisverzeichnis] assembliert alle .asm-Dateien
                                unterhalb eines Verzeichnisses (Vorgabe: ./) mit c82ass.sh. Bei
                                fehlerfreiem Lauf erscheint keine Meldung; bei einem Fehler wird die
                                Datei mit den Fehlerzeilen des Assemblers gemeldet.
                                Rückgabewert: 0 = alles fehlerfrei, 1 = Fehler
asm_aufraeumen.sh             # entfernt .log, .mqo, .obj und .sym (Trockenlauf; --loeschen löscht)
make_c82ass.bat               # Batch-Datei zum Erzeugen des Assemblers unter Windows (Pendant zu make_c82ass.sh)
Dokumentation                 # diese Dokumentation
_Linux                        # Linux-Binary des Assemblers
projekdetails                 # zuletzt benutzte Aufrufparameter
projekdetails.<Projekt>       # gesicherte Aufrufparameter je Projekt
Projekt_1                     # Projektverzeichnis (Beispiel, mit Unterverzeichnissen)
Sourcecode                    # C-Quellcode des Assemblers selbst
_Windows                      # Windows-Binary des Assemblers
```

```
./Assembler/Dokumentation:
C8205_Assembler_Dokumentation.md   C8205_Assembler_Dokumentation.odt   C8205_Assembler_Dokumentation.pdf

./Assembler/_Linux:
c82ass

./Assembler/Projekt_1:
… die Projekte, die zu einem Quelltext alle erzeugten Dateien zusammenhalten
```

Projektverzeichnisse können beliebig benannt sein, auch Unterverzeichnisse enthalten. In ihnen steht der Quelltext, die erzeugte Objektdatei (ein binär codierter R300-Lochstreifen, siehe 3.1) und weitere Hilfsdateien. Das Programm `c82ass` erwartet seine Dateien immer in einem Projektunterverzeichnis, das im gleichen Pfad wie es selbst liegen muss.

```
./Assembler/Sourcecode:
alle .c und .h Dateien des Assemblerquelltextes und die Makrodatei c82ass.makros

./Assembler/_Linux:
Der Pfad _Linux enthält alle Linux-Binaries und Libraries des Assemblers

./Assembler/_Windows:
Das Windows-Binary des Assemblers
```

Für die Prüfung von Assemblerergebnissen im Interpreter dient die Datei `.test` (s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a).

`asm_aufraeumen.sh` entfernt die erzeugten Dateien `.log`, `.mqo`, `.obj` und `.sym` unterhalb von `Assembler` (nicht in `Dokumentation`, `Sourcecode`, `_Linux`, `_Windows`); ohne Parameter wird nur angezeigt, was gelöscht würde (Trockenlauf), mit `--loeschen` wird tatsächlich gelöscht. Beide Skripte (`asm_alle.sh` und `asm_aufraeumen.sh`) gehören in das Verzeichnis `Assembler`.

2. Überblick

Sowohl das Programm `c82ass.sh` (Linux) als auch das Windows-Binary `_Windows\c82ass.exe` (die Datei `make_c82ass.bat` erzeugt es nur) werden über Aufrufparameter versorgt. Bei jedem Start eines der Programme werden die jeweils letztbenutzten Parameter automatisch in der Datei `projekdetails` abgelegt, sodass ein Folgestart ohne Angabe von Parametern gelingt. Zur Änderung müssen die Parameter einfach mit veränderten Werten wieder angegeben werden.

```
$ c82ass.sh -h
C8205 Assembler C82aas Version 1.00-038
Usage: ./_Linux/c82ass -p Projektverzeichnis -s Quellcode-Datei [-o Objektdatei] [-t Testdatei] [-m Makrodatei]
    [-WARN=SYM,SPR,TOT,ALLE] [--xref] [--ABSPR] [--color | --no-color]
       ./_Linux/c82ass -h | --help | --version

  -p Projektverzeichnis  Verzeichnis mit dem Quelltext; dorthin werden auch die Ausgabedateien
                         (.log .obj .mqo .sym .dmp .test) geschrieben. Ohne -p gilt der zuletzt
                         benutzte Wert aus der Datei 'projekdetails' im aktuellen Verzeichnis.
  -s Quellcode-Datei     Assembler-Quelltext im Projektverzeichnis; die Endung .asm darf entfallen.
                         Ohne -s gilt ebenfalls der zuletzt benutzte Wert aus 'projekdetails'.
  -o Pfad | Datei        Ablageort der .obj-Datei statt des Projektverzeichnisses. Verzeichnis
                         (existiert, oder endet auf / bzw. \): Dateiname aus -s mit Endung .obj.
                         Sonst Dateiname mit oder ohne Pfad (relativ zum aktuellen Verzeichnis),
                         unveraendert uebernommen. Das Verzeichnis muss existieren.
  -t Pfad | Datei        wie -o, aber fuer die .test-Datei (Endung .test).
  -m Makrodatei          Pfad der Makrodatei statt 'c82ass.makros' (sonst: aktuelles Verzeichnis,
                         ersatzweise 'Sourcecode/'). Mit -m wird nur dieser Pfad benutzt.
  -WARN=liste            Zusaetzliche Warnungen; Werte durch Komma getrennt (Gross-/Kleinschreibung egal):
                           SYM  Symbole, die definiert, aber nirgends verwendet werden.
                           SPR  Sprungbefehle, deren Ziel ein Datenwort (dw, da) oder ein unbelegtes Wort ist.
                           TOT  Befehle hinter einem unbedingten Sprung, auf die nichts verweist.
                           ALLE alle drei.
                         Die Meldungen stehen auch im .log (hinter der Symboltabelle).
  --xref                 Gibt nach fehlerfreiem Lauf eine Querverweisliste aus: je Symbol Wert,
                         Definitionszeile und alle Zeilen des Quelltexts, die es verwenden. Sie steht auch im .log.
  --ABSPR                Statt der .obj-Datei einen absolut adressierten, binaer im R300-Code codierten
                         Lochstreifen <name>.kep fuer das Kleine Eingabeprogramm (KEP) erzeugen:
                         Leitbefehl <start> 240: und Worte als Dualziffern (auch --ABSOLUTPROGRAMM, --ASBPR).
  --color, --no-color    Farbige Ausgabe erzwingen bzw. abschalten. Ohne Angabe nur, wenn STDOUT und
                         STDERR ein Terminal sind und die Umgebungsvariable NO_COLOR fehlt.
  -h, --help             Diese Hilfe ausgeben.
  --version              Die Versionsnummer ausgeben (steht auch in der ersten Ausgabezeile).
```

`c82ass` muss aus dem Verzeichnis `Assembler` gestartet werden; dort werden auch `projekdetails` und (zuerst) `c82ass.makros` gesucht (sofern nicht mit `-m` ein Pfad angegeben ist).

- `-p` nennt das Verzeichnis, in dem die `.asm`-Datei **direkt** liegt (z. B. `Projekt_1/bsp_5.16`). Unterverzeichnisse werden nicht durchsucht; ein Pfadanteil bei `-s` wird abgeschnitten.
- Hat der Name bei `-s` keine Endung aus genau drei Zeichen, wird `.asm` angehängt (`bsp_5_16` wird zu `bsp_5_16.asm`).
- `-o` und `-t` legen fest, wo die Objektdatei (`.obj`) bzw. die Testdatei (`.test`) abgelegt wird. Ohne diese Schalter liegen beide im Projektverzeichnis; der Dateiname wird aus `-s` gebildet. Ist ein Schalter angegeben, gilt statt des Standardorts der angegebene Pfad. Der Wert ist entweder ein Verzeichnis (es existiert bereits, oder der Wert endet auf `/` bzw. `\`) – dann wird der Dateiname aus `-s` gebildet (Name der Quellcode-Datei ohne `.asm`, dazu `.obj` bzw. `.test`) – oder ein Dateiname, mit oder ohne Pfad. Ein Dateiname wird unverändert übernommen (eine fehlende Endung wird nicht ergänzt); ohne Pfadanteil gilt er relativ zum aktuellen Verzeichnis, nicht zum Projektverzeichnis.
- Das Verzeichnis muss existieren; der Assembler legt es nicht an. Ein nicht vorhandenes Verzeichnis, ein Schalter ohne Wert (auch wenn der nächste Parameter mit `-` beginnt) und ein zu langer Pfad sind Fehler (Rückgabecode 16). `-o` und `-t` gelten nur für den einen Aufruf und werden nicht in `projekdetails` gespeichert; die übrigen Ausgabedateien (`.log`, `.mqo`, `.sym`, `.dmp`) bleiben im Projektverzeichnis. Beispiel: `$ c82ass.sh -p Projekt_1/bsp_5.16 -s bsp_5_16 -o ausgabe/ -t tests/bsp_5_16_neu.test`
- `-h` und `--help` geben den Usage-Text aus, `--version` nur die Versionszeile „C8205 Assembler C82aas Version 1.00-038“. Beide enden ohne Assemblerlauf mit Rückgabecode 0.
- `projekdetails` enthält die Zeilen `-p <Verzeichnis>` und `-s <Datei>`; fehlt einer der Parameter im Aufruf, wird der gespeicherte Wert benutzt.
- `-WARN=<liste>` schaltet zusätzliche Warnungen ein. Die Werte stehen durch Komma getrennt (z. B. `-WARN=SYM,SPR`); Groß-/Kleinschreibung ist egal, ein unbekannter Wert ergibt eine Warnung. Über `c82ass.sh` darf auch `--WARN=SYM` geschrieben werden. Alle Warnungen erscheinen erst nach fehlerfreiem Pass 3 und verändern weder `.obj` noch `.dmp`; sie lassen den Rückgabecode bei 0. In die `.log`-Datei werden sie hinter der Symboltabelle angehängt, ohne Farbcodes und mit denselben Überschriften wie auf STDOUT; ohne `-WARN=` und `--xref` bleibt die `.log` unverändert.
  - `SYM` meldet Symbole, die definiert, aber nirgends verwendet werden (vom Assembler erzeugte Symbole wie Literalsymbole und Makro-Labels werden nicht gemeldet).
  - `SPR` meldet Sprungbefehle (unbedingt, bedingt oder mit Rückkehrabsicht; Operationscodes 000–007, 024–027, 040–057, 064–077, 100–107, 124–127, 140–157, 170–177), deren Ziel kein Befehl ist, sondern ein Datenwort (`dw` mit Wert ungleich 0, `da`) oder ein Wort, das der Assembler gar nicht belegt. Nicht gemeldet werden Ziele, die als Befehl dienen können: `dz`, `dt`, `dto` und `dtm` erzeugen Ausgabebefehle, `db`, `dbn` und `dw 0` reservieren Zellen, in die das Programm zur Laufzeit etwa den Rücksprungbefehl schreibt. Ebenfalls nicht gemeldet wird ein Sprungbefehl, den ein Lade- oder Speicherbefehl anspricht (Adresskonstante in Befehlsform wie `000 feld`): Er gilt als Datenzelle. Sprünge mit absoluter Adresse (`abs`-Symbol) werden nicht geprüft.
  - `TOT` meldet Befehle, die unmittelbar hinter einem unbedingten Sprung (000–007, 100–107) stehen und auf die nichts verweist: keine Marke, die irgendwo verwendet wird, kein Operand mit relativer Adresse, kein `da`-Wort, nicht die Startadresse und nicht Adresse 0000. Folgen mehrere solche Befehle aufeinander, wird nur der erste gemeldet, mit der Zahl der folgenden. Nicht gemeldet wird ein unbedingter Sprung hinter einem unbedingten Sprung (Sprungtabelle, die das Programm durch berechnete Adressen anspringt; die C8205 kennt keinen indirekten Sprung) und ein Befehl hinter einem Sprung, den ein Lade- oder Speicherbefehl anspricht (das Programm überschreibt ihn zur Laufzeit).
  - `ALLE` schaltet `SYM`, `SPR` und `TOT` ein.
  - `SPR` und `TOT` sind Hinweise und keine Fehler: Programme, die Befehle zur Laufzeit in Datenworte schreiben oder Sprungadressen berechnen, können sie zu Recht auslösen. Die Meldungen haben die Form `<datei>:<zeile>: Warnung: …` auf STDERR; auf STDOUT stehen eine Überschrift und die Anzahl.
- `-m <Makrodatei>` gibt den Pfad der Makrodatei an, statt des festen Namens `c82ass.makros`. Mit `-m` wird nur dieser Pfad benutzt (keine Ersatzsuche in `Sourcecode/`). Lässt sich die Datei nicht öffnen, endet der Lauf mit Rückgabecode 16 und einer Meldung mit Pfad und Ursache; `-m` ohne Wert ist ebenfalls ein Fehler (16).
- `--xref` gibt nach einem fehlerfreien Lauf eine Querverweisliste aus: je Symbol Wert, Definitionszeile (Name in Spalte 1) und alle Zeilen des Quelltextes, in denen der Name als ganzes Wort vorkommt (Kommentare ab `;` zählen nicht). Vom Assembler erzeugte Symbole fehlen. Die Operanden aus Makroexpansionen stehen im Listing und zählen mit: ein Symbol, das ein Makro verwendet, gilt als verwendet in der Zeile des Makroaufrufs

(Definitionen im Quelltext und in der Makrodatei zählen nicht gesondert). Die Liste steht zusätzlich am Ende der `.log`-Datei. Ohne `--xref` entsteht keine Liste.

- `--color` und `--no-color` erzwingen bzw. unterdrücken die farbige Ausgabe. Ohne Angabe wird nur farbig ausgegeben, wenn STDOUT und STDERR ein Terminal sind und `NO_COLOR` nicht gesetzt ist; bei Umleitung in Datei oder Pipe stehen keine Farbcodes in der Ausgabe.

3. Der Assembler c82ass

3.1 Programmstart

Der Assembler `c82ass` erwartet eine Eingabedatei mit `<quelltext>` unter einem Dateinamen `.asm`.

- **Pass 0:** liest die Makrodatei `c82ass.makros` (Vorgabe `c82ass.makros`; gesucht zuerst im Arbeitsverzeichnis des Aufrufs, sonst in `Sourcecode/`; mit `-m` ist ein anderer Pfad wählbar), sammelt die im Quelltext definierten Makros (Abschnitt 4.1), prüft die Makrodefinitionen und löst die Makroaufrufe im Quelltext auf (Abschnitt 4).
- **Pass 1:** ermittelt die Literaltabelle, ausgegeben im Logfile `.log`.
- **Pass 2:** ermittelt die Symboltabelle, ausgegeben im Logfile `.log`, und schreibt die Symboldatei `.sym`.
- **Pass 3:** erzeugt aus dem aufgelösten Quelltext (Literale und Symbole eingesetzt) die Kennzeichenbefehle für die Eingabe mit Kennzeichen (Format des Minimalprogramms, nicht des KEP), ausgegeben im Objektfile `.obj`. Im selben Pass entstehen die Hilfsdatei `.mqo`, die formatierte Speicherbelegung `.dmp` und die Trommeldatei `.test` für den Interpreter.

Auf STDOUT gibt der Assembler den formatierten Quelltext (einschließlich der aufgelösten Literale) aus. Der Aufruf erfolgt zum Beispiel in der Form:

```
$ c82ass.sh -p PzBeispiele_4 -s bsp_5.17_Oktaldruck.asm
```

Meldungen mit `Fehler:`, `Warnung:` und `Abbruch` sowie die Statuszeilen `Projektverzeichnis:`, `Benutze für -s die Datei:`, `Leitadresse:` (nur wenn der Quelltext das Pragma `LEITADRESSE` enthält, siehe 3.3.12) und `Startadresse:` gehen auf STDERR (farbig hervorgehoben). Fehler werden je nach Pass in unterschiedlicher Form gemeldet (siehe die folgende Tabelle); jeder Fehler beendet den Lauf mit Rückgabecode 16. Fehlermeldungen mit Ort erscheinen in der Form `<datei>:<zeile>: Fehler: <text>`. Bei einem Fehler werden keine `.obj`-, `.mqo`- und `.test`-Datei hinterlassen (eine unvollständige Objektdatei könnte sonst versehentlich geladen werden); `.log`, `.sym` und `.dmp` bleiben zur Fehlersuche erhalten. Meldungen und Statuszeilen sind nur auf einem Terminal farbig (siehe `--color`).

Fehlerobergrenze und Zusammenfassung (ab Version 1.00-034): Der Assembler gibt höchstens 20 Fehlerzeilen aus, in allen Pässen und für alle Fehlerarten, auch für „bei Operand“, „bei Labeldefinition“, „bei Macrodefinition“ sowie für Fehler in `dt` und `dz`. Dieselbe Meldung (gleicher Text, ohne Zeilennummer) erscheint höchstens dreimal und nur mit verschiedenen Quellzeilen; jede weitere Wiederholung wird nur gezählt. Neue Meldungen jenseits der 20 Zeilen werden ebenfalls nur gezählt. Am Programmende, auch nach einem Abbruch, folgt für jede zusammengefasste Meldung eine Zeile „`<datei>: <n>-mal gleiche Fehlermeldung (Zeile <z>, davon <k> einzeln ausgegeben): <text>`“ (bei verschiedenen Zeilen „Zeilen `<a>` bis `<b>`“) und, wenn etwas zusammengefasst oder unterdrückt wurde, die Gesamtzeile „`<datei>: <n> Fehler insgesamt, <w> gleiche Wiederholungen zusammengefasst, <u> weitere Fehlermeldungen unterdrückt (Obergrenze 20 Fehlerzeilen).`“; die Teile für `w` und `u` entfallen, wenn sie 0 sind. Die Meldungen „Abbruch …“ sind nicht begrenzt. Der Rückgabecode bleibt 16.

Fehlerausgaben

Fehlerart	Ausgabe auf STDOUT (Listing)	Ausgabe auf STDERR	Ablauf danach
Pragma `LEITADRESSE` fehlerhaft (beim Einlesen des Quelltextes, vor Pass 0; siehe 3.3.12)	–	`Fehler: Zeile <n> der Quelldatei: Das Pragma LEITADRESSE …`, `Fehler: Abbruch.`	Abbruch, keine Ausgabedateien
Makrodefinition fehlerhaft (Pass 0)	–	`Fehler: Makrodefinition …`, `Fehler: N fehlerhafte Makrodefinition(en) …`	Abbruch nach Pass 0, keine Ausgabedateien

Fehlerart	Ausgabe auf STDOUT (Listing)	Ausgabe auf STDERR	Ablauf danach
Makroaufruf fehlerhaft (Pass 0), z. B. Parameteranzahl oder -typ	`;*** Fehler: erwartet 2 Parameter, aber 1 wurden übergeben!` bzw. `;*** Fehler $1 Parameter $1: Wert "123" …`	`Warnung: Compilierung endete mit FEHLERn.`	Abbruch nach Pass 0, keine Ausgabedateien
Syntaxfehler in Pass 1 (z. B. `#` am Anfang eines `dt`-Textes)	`;*** 1 Fehler: …`	`Fehler: bei Operand: … in Zeile: …`, `Fehler: Abbruch wegen Syntaxfehler.`	Pass 2 läuft noch (`.log`, `.sym`), Pass 3 entfällt
Fehler in Pass 2 (z. B. Symbol doppelt, Programmzählerüberlauf über 7777)	`;*** (2) Fehler: …`	`Fehler: Abbruch wegen Syntaxfehler.`	Pass 3 entfällt
Fehler in Pass 3 (Operand oder Operationscode ungültig)	Zeile `<adresse>: EE …` mit darunter stehender Meldungszeile `bei Operand: … ungültig`	`<datei>:<zeile>: Fehler: Operationscode … ist ungültig` bzw. `Symbol … ist nicht definiert`; `Fehler: Abbruch. Keine Objekt-, mqo- und Testdatei erzeugt …`	alle Pässe gelaufen; keine `.obj`, `.mqo`, `.test`; `.log`, `.sym`, `.dmp` bleiben
Belegungsfehler in Pass 3	`;*** 3 Fehler: Adresse … doppelt belegt: …`	`Fehler: Adresse … doppelt belegt: …`, `Fehler: Abbruch.`	wie Fehler in Pass 3

Belegungsprüfung (Pass 3): Jedes Wort der Trommel darf nur von einer Anweisung beschrieben werden. Geprüft werden Maschinenbefehle, `dw`, `da`, `dbn`, `dt`, `dto`, `dtm`, `dz` und die automatisch erzeugten Literale. Überschneidet sich die fortlaufende Speicherbelegung mit einem früher belegten Bereich, zum Beispiel weil ein `equ` zurückspringt oder das Programm in einen mit `equ` weiter hinten angelegten Datenbereich hineinläuft, meldet der Assembler `Adresse <oktal> doppelt belegt: "<neue Zeile>" ueberschneidet sich mit "<frühere Zeile>".` und der Lauf endet mit Rückgabecode 16. Pro Zeile wird höchstens ein Konflikt gemeldet. Nicht geprüft werden `db` (reserviert nur Speicherplätze), `equ`, `start`, `abs` und `end`.

Die `<adresse>` der Befehle im Objektfile beginnt immer bei 0000. Ein Operand von `start` ändert die Adressen nicht; der Wert steht nur in der Kennzeichenzeile `210000<zahl>` (siehe 3.3.7).

Der Assembler erzeugt bzw. bearbeitet folgende Dateien im Verzeichnis der `.asm`-Datei (Beispiel `Projekt_1/bsp_5.16/`):

Endung	Inhalt
`.asm`	Quelltext (Eingabe)
`.dmp`	formatierte Speicherbelegung (Listing: Adresse, Maschinenwort, Label, Mnemonic, Operand, Kommentar); entspricht dem Listing auf STDOUT, jedoch ohne die Farbcodes der Fehlermeldungen; das Kennzeichen 01 kann durch das Pragma `LEITADRESSE` ersetzt sein (siehe 3.3.12)
`.log`	Abschnitte, soweit vorhanden: Literaltabelle, EQU-Definitionen, ABS-Definitionen, Symboltabelle (Symbol, RelAddr, Adresse, original Operation) und Adresstabelle (nach Adresse sortiert)
`.mqo`	aufbereiteter Quelltext. Gegenüber dem Quelltext fehlen `start`, `end`, Makrodefinitionen, reine Kommentarzeilen und `abs`-Anweisungen; Makroaufrufe sind durch ihre Ersetzung ersetzt, `equ`-Operanden aufgelöst, und die Literale stehen am Ende als `dw`-Zeilen (`l0001 dw 5`)
`.obj`	binär codierter R300-Lochstreifen mit dem Objektcode K1K2 (Eingabe mit Kennzeichen); es gibt nur noch das binäre Format: je Zeichen eine Zeile aus acht Ziffern 0/1 (Spurfolge w v u p 8 4 2 1, 1 = gelocht, ungerade Lochzahl, Kodetabelle 1, Klbuch). Der Text der Objektzeile steht ohne Leerzeichen und wird mit `.` abgeschlossen, zum Beispiel `010007000.` für das Listing `01 0007 000`; ein Zeilenende wird nicht gelocht (ein Lochstreifen ist endlos, SatzMarke/WRZL würde das Minimalprogramm v6 als zusätzliches Wort 0 speichern), die Wörter folgen unmittelbar aufeinander, das letzte Zeichen ist `/`. Die Datei kann unmittelbar in den Ordner des Lochbandlesers gelegt werden; zurück in Text wandelt `c8205_lochstreifen_umwandlung.py` mit `--IN-TXT`. Das Kennzeichen 01 kann mit dem Pragma `LEITADRESSE` durch 00 oder 02 … 09 ersetzt werden (siehe 3.3.12); bei 00 entfällt die Zeile `210000<zahl>`
`.kep`	nur mit dem Schalter `--ABSPR`, **statt** `.obj`: absolut adressierter, binär im R300-Code codierter Lochstreifen für das Kleine Eingabeprogramm (KEP); siehe 3.1a

Endung	Inhalt
`.sym`	Symboldatei. Je Zeile `<symbol>,<adresse>` (vierstellig oktal, in Quelltextreihenfolge, einschließlich Literalsymbolen wie `l0001`)
`.test`	Trommeldatei im Format des Interpreters (`Adresse: Wert # Quelltextzeile`) mit Kopf (Abschnitt `Trommeldatei:`) und Fuß (Abschnitt `Tastenfolge:` mit `H G R`). Der Kopf besteht aus den Vorspannzeilen des Quelltextes (siehe 3.1b); enthält der Quelltext keine, erzeugt der Assembler als Platzhalter `#DOK-0001`, `#SPEI-0001 0000-0000` und `#PROT STUFE=2`. Kopf und Fuß müssen für den jeweiligen Fall ergänzt bzw. erweitert werden. Zeilen gibt es nur für Wörter mit K1=00 oder 01 (Befehle, `dw`, `da`, `dz`, `dt`, `dto`, `dtm`, Nullwort); `db`, `equ`, `start` und `abs` erzeugen keine Zeile; Grundlage für Testaufträge, s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a

Vor jedem Lauf löscht der Assembler die Dateien `.log`, `.obj`, `.mqo`, `.sym`, `.test` und `.dmp` des Projekts und erzeugt sie neu. Mit `-o` bzw. `-t` (Abschnitt 2) wird statt der `.obj`- bzw. `.test`-Datei im Projektverzeichnis die dort angegebene Datei gelöscht und neu erzeugt; eine ältere Datei im Projektverzeichnis bleibt unberührt. Die früher genannten Endungen `.mqt` (modifizierter Quelltext) und `.l82` (Objektcode für den `C8205Simulator`) erzeugt der aktuelle Assembler nicht mehr. Dasselbe gilt für `.mac`: Der Quellcode legt keine solche Datei an.

3.1a Schalter `--ABSPR` (absolut adressierter KEP-Lochstreifen)

Das KEP liest nur Dualziffern. Mit diesem Schalter kann der Assembler statt einer `.obj`-Datei im BIS-Minimalprogramm-Format eine binär codierte Datei im KEP-Format erzeugen.

Mit `--ABSPR` (gleichwertig `--ABSOLUTPROGRAMM` und `--ASBPR`, Groß-/Kleinschreibung egal) schreibt der Assembler deshalb statt der `.obj`-Datei die Datei `<name>.kep` (die mit `-o` wie bei `.obj`-Dateien auch in den gewünschten Pfad umlenkbar ist).

Sie ist eine binäre Lochstreifendatei, d. h. je Zeile wird ein einzelnes Zeichen durch die 8 Bit des R300-Codes dargestellt (in der Form der Lochstreifen-Hilfsprogramme: je Lochungszeile 8 Ziffern 0/1 (Spurfolge `w v u p 8 4 2 1`), nur die Zeichen `0 1 : .`). Eine solche Datei ist also ohne weitere Umwandlung in den Leser 1 einlegbar.

Der Aufbau wird aus den Zeilen, die sonst in eine `.obj`-Datei kämen, abgeleitet; die Kennziffern K1K2 entfallen, bestimmen aber die Aufbereitung.

Automatisch wird ein Leitbefehl vorangestellt: `<start> 240:`. Die Adresse kommt aus der `start`-Anweisung (oktal, ohne Vornullen, höchstens 8 Oktaltriaden), dann folgt der Operationscode 240.

Dabei werden die K1K2 wie folgt umgesetzt:

K1K2	Umsetzung
`00`	Rest der Zeile ohne Leerzeichen unverändert, mit `.` abgeschlossen
`01`	Adressstellen plus aktueller Wert `oo..oo` (Anfangswert = `start`), dann die 3 Oktalziffern des Operationscodes, ohne Vornullen, mit `.`
`20`	`oo..oo` wird durch den angegebenen Wert (ohne Leerzeichen) ersetzt und gilt für spätere Additionen; dazu ein neuer Leitbefehl `<wert> 240:`
`21`, Endezeichen `/`	entfallen

Jede Oktalziffer wird durch ihre 3 Bits ersetzt, führende Nullbits entfallen (Beispiele: `0040 240:` = `100000010100000:`, `0001 164.` = `1001110100.`). Das Pragma `LEITADRESSE` hat Vorrang und wird vor der Umsetzung ins binäre Format angewendet: Ersetzt es 01 durch 00, wird K1=00 behandelt (Rest unverändert, keine Addition). Ersetzt es 01 durch 02 … 09, werden diese Zeilen nicht umgesetzt (sie fehlen in der `.kep`); der Assembler gibt dazu eine Warnung mit der Anzahl aus. Gibt es danach keine Zeile mit K1=01 mehr, gibt er eine Warnung aus; ist die Ursache das Pragma `LEITADRESSE 00` (absolut geschriebener Quelltext, der Regelfall beim Minimalprogramm), ist es nur ein Hinweis „Der Quelltext gilt als absolut adressiert, es wird keine Adresse umgerechnet“ (ab Version 1.00-030). Dasselbe gilt für `LEITADRESSE 00` zusammen mit `start <adresse>` ungleich 0: Der erste Leitbefehl lautet dann `<adresse> 240`, die Operanden bleiben aber unverändert (die Startadresse wird nicht addiert); der Assembler warnt (ab Version 1.00-028). Regel: Relativ geschriebener Quelltext mit `start <adresse>` ohne Pragma; absolut geschriebener Quelltext mit Pragma `LEITADRESSE 00` und `start` ohne Operand. Wird ein Wort auf eine Zelle von 0002 bis 0037 geladen (dort liegt das KEP selbst), gibt der Assembler eine Warnung mit der Zelle aus (ab Version 1.00-029; vorher warnte er bei jedem Leitbefehl auf 0000 bis 0037). Die Zellen 0000 und 0001 sind erlaubt und warnen nicht: Dort trägt das geladene Programm, wie das Minimalprogramm, seinen Sprung zur Startadresse ein; der Bahnsperrenschalter muss beim Laden aus sein. Das KEP selbst ist nicht Teil des Lochstreifens.

3.1b Vorspann der `.test`-Datei (Testmarken im Quelltext)

Der Assembler erzeugt immer eine `.test`-Datei. Die „#…“-Makros zur Teststeuerung mit dem Abschnitt `Trommeldatei:` (Kopf), den Trommelzeilen und dem Abschnitt `Tastenfolge:` (Fuß) können schon im Assembler-Quelltext als Assembler-Kommentare angelegt werden.

Regel. Der Assembler liest den Quelltext Zeile für Zeile und behandelt jede Zeile so:

1. Er überspringt Leerzeichen und Tabulatoren am Zeilenanfang. Das erste übrige Zeichen muss ein Semikolon `;` sein, sonst ist die Zeile keine Vorspannzeile. Ein Kommentar hinter einem Befehl (`ADD 5 ;#DOK-0001 …`) wird deshalb nicht übernommen.
2. Er überspringt das `;` und den Leerraum dahinter.
3. Der Rest muss mit einer Testmarke beginnen (siehe Tabelle). Sonst bleibt die Zeile ein gewöhnlicher Kommentar.
4. Er schneidet Leerraum und Zeilenende am Ende der Zeile ab und schreibt den Rest unverändert in die `.test`-Datei. Das `;` und der Leerraum davor erscheinen dort nicht.

Die Zeilen kommen in der Reihenfolge des Quelltextes in den Abschnitt `Trommeldatei:`. Sie dürfen an beliebiger Stelle des Quelltextes stehen, üblicherweise am Anfang. Zeilen mit mehr als 2047 Zeichen werden ignoriert.

Erkannte Testmarken.

Marke	Erkennung	Bedeutung
`#DOK-xxxx`	als Präfix	Dokumentationstext zu einer Debug-Nummer
`#SPEI-xxxx`	als Präfix	Speicherbereich ssss-eeee zur Ausgabe
`#SPUR-xxxx`	als Präfix	Speicherausgabe nach Spuradresse
`#PROT`	ganzes Wort	Protokollstufe (`#PROT STUFE=n`)
`#AC`, `#BZ`	ganzes Wort	Anfangswerte von Akkumulator und Befehlszähler (`#AC=…`, `#BZ=…`)
`#STARTE`	ganzes Wort	Ansprung eines Speicherplatzes; ersetzt die Tastenfolge (siehe unten)
`#TROMMEL`	ganzes Wort	Trommel leeren oder mit Abbild belegen (`#TROMMEL=LEER`, `#TROMMEL=MP`)
`--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe`, `--k0`	ganzes Wort	Geräteschalter mit Pfad (`--k0 datei=PFAD`)

Ein Eintrag mit „ganzes Wort“ gilt nur, wenn hinter der Marke Zeilenende, Leerraum oder `=` folgt. `#PROTOKOLL` allein ist also keine Testmarke. Die Bedeutung der Marken beschreibt `C8205-Interpreter-Durchführung_eines_Tests.md` in den Abschnitten 2, 2b, 2e und 2g.

Nicht übernommen (die Zeile bleibt ein gewöhnlicher Kommentar):

Zeile	Grund
`--pruefe-…`	gehört in den Abschnitt `Prüfungen:`, nicht in den Kopf
`#INTERRUPT`	muss zwischen zwei Befehlszeilen der Trommeldatei stehen, nicht im Vorspann
`;#; LEITADRESSE xx`	Pragma für den Objektcode (3.3.12), keine Testmarke
jede andere Kommentarzeile	keine Testmarke

Sonderfälle.

- *Keine Vorspannzeile:* Enthält der Quelltext keine einzige übernommene Zeile, erzeugt der Assembler die Platzhalter `#DOK-0001`, `#SPEI-0001 0000-0000` und `#PROT STUFE=2`. Schon eine einzige Vorspannzeile unterdrückt alle drei Platzhalter. Der Kopf enthält dann nur Ihre Zeilen.
- `#STARTE`*:* Steht `#STARTE` im Vorspann, ersetzt es die Tastenfolge. Der Assembler erzeugt dann keinen Abschnitt `Tastenfolge:` (sonst steht dort `H G R`).

Beispiel. Im Quelltext:

```
;#DOK-0001 Druck von BZ und AC
; --sw-ausgabe virtuelle_Geraete/SW-Ausgabe
;#PROT STUFE=2
;#; LEITADRESSE 03
; gewöhnlicher Kommentar
```

ergibt in der `.test`-Datei:

```
Trommeldatei:
#DOK-0001 Druck von BZ und AC
--sw-ausgabe virtuelle_Geraete/SW-Ausgabe
#PROT STUFE=2
0000: ... (die Zeilen aus dem Quelltext)
Tastenfolge:
H G R
```

3.1c Nachspann der `.test`-Datei (Prüfungs-Testmarken im Quelltext)

Durch die Definition von:

```
;#PRUEFE-START
… beliebiger Text ..
;#PRUEFE-ENDE
```

können die in `C8205-Interpreter-Durchführung_eines_Tests.md` (Abschnitt 3a) beschriebenen Prüfungen schon im Assembler-Quelltext definiert werden. Die Zeilen zwischen den beiden Marken werden am Ende der `.test`-Datei unter einer Zeile „`Prüfungen:`“ angefügt; das führende Semikolon (und der Leerraum dahinter) wird bei jeder Zeile dazwischen entfernt.

`;#PRUEFE-START` und `;#PRUEFE-ENDE` selbst werden nicht übernommen; dafür steht vor den Prüfungen die Zeile:

```
Prüfungen:
```

vorangestellt.

Weitere Regeln: Der Abschnitt steht hinter `Tastenfolge:` (bei `#STARTE` hinter den Trommelzeilen). Mehrere Bereiche ergeben einen gemeinsamen Abschnitt. Zeilen im Bereich ohne führendes Semikolon und Zeilen, die danach leer sind, werden übergangen. Fehlt `;#PRUEFE-ENDE` oder steht es ohne `;#PRUEFE-START`, gibt der Assembler eine Warnung aus; ein Bereich ohne Ende wird nicht übernommen.

Beispiel (`bsp_5_19_2_teilwoerter2.asm`). Im Quelltext:

```
;#PRUEFE-START
; BZ=3054 AC=77760000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
; 3000: 52526317434
; 3001: 52520000000
;#PRUEFE-ENDE
```

ergibt am Ende der `.test`-Datei:

```
Tastenfolge:
H G R
Prüfungen:
BZ=3054 AC=77760000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
3000: 52526317434
3001: 52520000000
```

3.2 Sprachdefinition

```
<zahl>              ::= <oktalzahl> | <dezimalzahl>
<oktalzahl>          ::= oktalziffer==0 [ oktalziffer [ oktalziffer … ]]
<oktalziffer>        ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
<vorzeichen>         ::= -
<dezimalziffer>      ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<dezimalzahl>        ::= [<vorzeichen>] dezimalziffer<>0 [ dezimalziffer [ dezimalziffer … ]]
<alphazeichen>       ::= abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
<sonderzeichen>      ::= % : + - / ` , . < " [ > ' ~ ] # * ( = ; ) ? !
<wert>               ::= <oktalzahl> | <dezimalzahl> | <symbol>
<rechenzeichen>      ::= + | - | * | /
<speicherplatz>      ::= Speicherstelle auf dem Trommelspeicher, fortlaufend nummeriert durch
                         eine Oktalzahl mit dem Wertebereich 00000 bis 07777.
<symbol>             ::= <adresssymbol> | <literalsymbol> | (leer)
<adresssymbol>       ::= Zeichenfolge aus 1 bis 32 Zeichen:
                         <alphazeichen> [ <alphazeichen> | <dezimalziffer> [ … ]],
                         abgeschlossen durch Doppelpunkt.
                         Sie wird vom Programmierer frei definiert. Der Unterstrich `_` wird ebenfalls akzeptiert.
                         Das erste Zeichen ist ein Buchstabe oder `_`, keine Ziffer. Erlaubt sind nur die
    ASCII-Buchstaben
                         `a`–`z`, `A`–`Z`, die Ziffern `0`–`9` und `_`; Zeichen wie `-`, `.` oder `ä` sind
    unzulässig.
                         Doppelt definierte Symbole, Symbole über 32 Zeichen und unzulässige Symbole sind Fehler.
<literalsymbol>      ::= 'l' (kleines L) gefolgt von einer Oktalziffernfolge von genau 4
                         Oktalziffern, abgeschlossen durch Doppelpunkt.
                         Sie wird vom Assembler automatisch aus dem Inhalt des Literalzählers vergeben.
<ausdruck>           ::= <werteausdruck> | <literal>
<operand>            ::= <werteausdruck> | <literal> | (leer)
<adressausdruck>     ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
                         Einschränkungen: kein negatives Vorzeichen;
                         Wertebereich nur von 00000 … 07777.
                         Ein Vorzeichen am Anfang (auch `+`) ist nicht zulässig.
                         Verstöße meldet der Assembler als Fehler in Pass 3.
<werteausdruck>      ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
                         Einschränkung: Bei `dw` und in `#`-Literalen ist ein relatives Ergebnis (eine einzelne
    symbolische
                         Adresse, auch mit Zahl) nicht zulässig. Zulässig: Zahlen, `abs`-Symbole, die paarweisen
                         Differenzen symbolischer Adressen (auch mehrere). Wertebereich -2^32 … 2^33-1.
                         Adressen werden mit `da` oder `@`-Literalen geschrieben.
<literal>            ::= #<werteausdruck> | @<adressausdruck>
                         Literal bedeutet, dass der numerische Wert von <werteausdruck> oder
                         <adressausdruck> auf einem vom Assembler selbst festgelegten
                         <speicherplatz> gespeichert wird. Diesem Speicherplatz wird vom
                         Assembler ein <literalsymbol> zugeordnet.
<adresse>            ::= numerischer Wert eines <symbol>, der einen <speicherplatz> adressiert.
<kommentar>          ::= ; (Semikolon) gefolgt von ASCII-Zeichen. | (leer)
                         Innerhalb der \-Zeichen von dt wird ; nicht als Kommentar betrachtet.
<blockkommentar>     ::= ```   als beginn auf einer Zeile für sich, ab Spalte 1
                         ```   als Ende auf einer Zeile für sich, ab Spalte 1
                         Dazwischen beliebiger Text
<trenner>            ::= Tabulator[<trenner>[...]] | Leertaste<trenner>[...]]
<befehl>             ::= [<symbol>] <trenner><operationscode> [<trenner><operand>]
                         [<trenner><kommentar>]
                         Alle Befehle müssen auf einer Zeile stehen; eine Zeile darf höchstens 255 Zeichen lang
    sein.
                         Ein <symbol> darf auch allein auf einer Zeile stehen (ohne <operationscode>). Die Zeile
    belegt keinen
                         Speicher; das Symbol erhält die Adresse der nächsten Zeile, die Speicher belegt, und der
    Befehlszähler
                         bleibt unverändert. Mehrere solche Symbole in Folge erhalten dieselbe Adresse. Vor start
    ist das ein
                         Fehler, wie ein Symbol auf der start-Zeile selbst.
<operationscode>     ::= <maschinenbefehl> | <assembleranweisung>
<assembleranweisung> ::= start | end | db | dbn | dw | da | dt | dto | dtm | dz | equ | equ10 | equ40 | equ100 |
    abs
<maschinenbefehl>    ::= <oktalziffer><oktalziffer><oktalziffer>
<quelltext>          ::= <befehl>[<befehl> [ … ]]
```

Auswertung von Ausdrücken (`<werteausdruck>` und `<adressausdruck>`): `*` und `/` binden stärker als `+` und `-`; Operatoren gleichen Ranges werden von links nach rechts ausgewertet (`2+3*4` ergibt 14, `10-2-3` ergibt 5). `/` ist eine ganzzahlige Division, deren Ergebnis zur Null hin abgeschnitten wird (`7/2` ergibt 3, `-7/2` ergibt -3); eine Division durch 0 ist ein Fehler. Ein Vorzeichen ist nur am Anfang des Ausdrucks zulässig (`-7/2`, `+5`, nicht `7/-2` oder `--5`). Klammern und Leerzeichen innerhalb eines Ausdrucks sind nicht zulässig.

Keine mnemonischen Operationscodes: Seit 1.00-026 kennt der Assembler nur noch dreistellige Oktalcodes (000–777). Die frühere Ansatztabelle mit `add`, `sub`, `ld`, `jmp`, `stopp` und `nop` (vorläufige Platzhalterwerte) ist entfallen; solche Namen ergeben jetzt den Fehler „Operationscode ist ungültig“.
```

3.3 Assembleranweisungen

3.3.1 Definiere Bereich

```
[<symbol>]  db  <zahl>
```

Hält <zahl> Speicherplätze frei, ohne ihren Inhalt zu definieren. <zahl> muss eine Oktalzahl (führende 0) oder eine Dezimalzahl sein, höchstens 010000. Negative Zahlen, Ausdrücke (z. B. 3+1), Symbole und ein fehlender Operand sind Fehler.

3.3.2 Definiere Bereich auf Null

```
[<symbol>]  dbn  <zahl>
```

Hält <zahl> Speicherplätze frei, setzt ihren Inhalt auf 0. Für <zahl> gilt dasselbe wie bei db. Je Speicherplatz erzeugt dbn in der Objektdatei die Zeile 000..

3.3.3 Definiere Wert

```
[<symbol>]  dw  <werteausdruck>
```

- Speichert den numerischen Wert von <werteausdruck> auf dem <speicherplatz>. Zulässig ist der Bereich -2^32 bis 2^33-1 (33-Bit-Wort), sonst meldet der Assembler Operand ist ungültig. Negative Werte werden im Zweierkomplement gespeichert (dw -5 ergibt 77777777773). Ein Ergebnis, das ein relativer Wert ist, ist im <werteausdruck> von dw nicht zulässig; das meint eine einzelne symbolische Adresse. Dagegen sind paarweise Differenzen symbolischer Adressen zulässig, auch mehrfach. Auch absolute Symbole (abs) sind zulässig; Adressen schreibt man mit da.
- Es wird keine Hauptleitadresse addiert.
- In der Objektdatei steht ein Datenwort als 00<elf Oktalziffern>.; nur der Wert 0 wird kurz als 0. ausgegeben (Nullwort ohne Kennzeichen).

3.3.4 Definiere Adresse

```
[<symbol>]  da  <adressausdruck>
```

Speichert den numerischen Wert von <adressausdruck> auf dem <speicherplatz>, verschoben um «9 in den Adressstellen. Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.

Das Ergebnis von <adressausdruck> muss bei da (und bei @-Literalen) immer eine **relative Adresse** sein. Jedes relative Symbol (Marke, equ-Symbol, Literalsymbol) zählt dabei +1, wenn es addiert wird, und −1, wenn es subtrahiert wird; Zahlen und abs-Symbole zählen 0. Es muss mindestens ein relativer Adressanteil übrig bleiben (Summe ≥ 1). Unzulässig sind daher:

- Konstanten und konstante Ausdrücke (da 5, da absx, da absx+2),
- paarweise Differenzen relativer Adressen, die sich aufheben (da a-b, da b-a+absx),
- relative Adressen in * oder / (da a*2).

Zulässig sind z. B. da a, da a+5, da a-b+c, da a+absx und da *. Verstöße meldet der Assembler in Pass 2 als Fehler mit Datei und Zeile (Operand: "a-b" ist bei da nicht zulässig: …). Konstante Werte schreibt man mit dw.

```
[<symbol>]  da  <literal>
```

- Erzeugt am Programmende automatisch aus <literal> eine Definition der Art <literalsymbol> da <adressausdruck>.
- Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.
- Nimmt stattdessen die <adresse> von <literalsymbol> in den Befehl, der das <literal> enthält.

3.3.5 Definiere Text zur Ausgabe von Zeichen

```
[<symbol>]  dt[o|m]  \<alphazeichen> | <leerzeichen> | <dezimalziffer>
             <sonderzeichen> [ <alphazeichen> | <leerzeichen> | <dezimalziffer> | <sonderzeichen>[...]]\
```

- Gespeichert wird ein Code gemäß R300-Code Ausgabeadresse C8205 (siehe R300-code.pdf). Die Abspeicherung erfolgt als Folge von Speicherplatzinhalten in der Form (oktal): `0000 xxxx 030`, wobei `xxxx` die Ausgabeadresse ist.
- Die Zeichen „\" am Anfang und Ende sind erforderlich, wenn der Text mit `#` beginnt, ein `;` enthält (sonst beginnt dort der Kommentar) oder führende bzw. abschließende Leerzeichen enthalten soll. Leerzeichen innerhalb des Textes (`dt a b`) sind auch ohne „\" zulässig; sonst werden führende und abschließende Leerzeichen entfernt. Ein führendes `#` kennzeichnet ein Literal (`#<werteausdruck>`), keinen Text; `dt #ab` und `dto #x` meldet der Assembler als Fehler. Richtig ist `dt \#ab\` bzw. `dto \#x\`. Ein `#` innerhalb des Textes (`dt a#b`) ist zulässig.
- `dt` ist für längere, zusammenhängende Textausgaben gedacht. Sein Vorteil: Der Text lässt sich ohne ausgeschriebene Umschaltzeichen definieren, der Quelltext bleibt gut lesbar. Für verstreute Einzelzeichen ist `dt` nicht geeignet, dafür nimmt man `dz` (3.3.6) und schreibt alle Umschaltzeichen explizit.
- Der Assembler setzt `dt` zur Assemblerzeit in Maschinenbefehle um und kennt den Zustand KB/GB zur Laufzeit nicht. Er führt den Zustand nur in der Reihenfolge des Quelltexts weiter. Beim allerersten `dt` des Quelltexts nimmt er KB an. Jedes weitere `dt` setzt den Zustand des vorangehenden `dt` fort und fügt nur dort ein Umschaltzeichen ein, wo sich der Zustand innerhalb des Textes ändert.
- Daraus folgt: Vor jeder zusammengehörenden Folge von `dt`-Anweisungen muss ein Umschaltbefehl durchlaufen werden, der exakt auf den Zustand des ersten Zeichens dieser Folge schaltet (`dz $inkb`, `dz $ingb` oder `dtm` als erste Anweisung der Folge). Das gilt für jede Folge neu. Es kann dabei ein eigentlich unnötiger Umschaltbefehl entstehen. Der ist beabsichtigt, weil sich Assemblerzeit und Laufzeit nicht synchronisieren lassen. Umschaltzeichen mit `dz` zwischen zusammengehörenden `dt`-Anweisungen sind nicht zulässig, sie bringen den Zustand des Assemblers durcheinander.
- Praktischer Hinweis dazu: `dtm` fügt immer vor dem ersten Zeichen ein Umschaltzeichen passend zum ersten Buchstaben ein. Damit ist es die natürliche erste Anweisung einer `dt`-Folge. Es ersetzt das von Hand gesetzte `dz $inkb` oder `dz $ingb` davor.
- `dto` verhält sich wie `dt`, außer dass generell KEINE Umschaltzeichen automatisch eingefügt werden. Das bedeutet, in einer `dto`-Anweisung kann man KEINE gemischte Groß- und Kleinschreibung definieren (auch keine Zahlen und Großbuchstaben usw.).
- `dtm` verhält sich wie `dt`, außer dass vor dem ersten Zeichen generell ein Umschaltzeichen, dem ersten Buchstaben entsprechend, automatisch eingefügt wird.

3.3.6 Definiere Zeichen (spezielle Sonderzeichen)

```
[<symbol>]  dz  $wrzl | $sm | $inkb | $ingb | $irrz | $irrs | $trans | $bm | $vorl |
                $irrb | $wr | $tab | $zr | % | : | + | ` | / | , | . | -
```

- Gespeichert wird ein Code gemäß R300-Code Ausgabeadresse C8205 (siehe Tabelle 2 unten). Die Abspeicherung erfolgt auf einem Speicherplatz in der Form (oktal): `0000 ssss 030`. Eine automatische Ergänzung von Umschaltzeichen wie bei `dt` erfolgt hier generell NICHT.
- Findet man z. B. `dz $wrzl` zu kompliziert, kann man das mit einem Makro vereinfachen:

```
WRZL:  MACRO
           dz  $wrzl  ; Zeilenschaltung
       MEND
```

  (In der mitgelieferten `c82ass.makros` enthält `WRZL` zusätzlich die Anweisung `ANWAHL 4700`.)
- Das Zeichen `;` kann bei `dz` nicht angegeben werden, weil es einen Kommentar einleitet; der Operand für den Code 0150 ist `:` (vgl. Tabelle 2 und 3).

3.3.7 Beginn des Quellcodes

```
start  [<zahl>]
```

- Nur als 1. Anweisung im `<quelltext>` zulässig, und nur einmal; ein Label vor `start` ist ein Fehler. Der Operand muss eine Konstante sein. Der Befehlszähler beginnt unabhängig vom Operanden bei 0. Dasselbe gilt für ein Label allein auf einer Zeile vor `start`. Eine Maschinencodeausgabe erfolgt nicht; mit Operand wird im Objektcode die Kennzeichenzeile `210000<zahl>` ausgegeben.
- Ist `<zahl>` angegeben, wird die 1. Leitadresse in den „Lochstreifen" ausgegeben. Es wird dann auf diese 1. Leitadresse + Ladeadresse geladen. Die Ladeadresse existiert dabei nur beim Laden.

- Das Kennzeichen 01, das für die 1. Leitadresse steht, kann mit dem Pragma `;#; LEITADRESSE xx` durch ein anderes Kennzeichen ersetzt werden (Abschnitt 3.3.12). Das Pragma muss vor `start` stehen.
- Mit `;#; LEITADRESSE 00` wird dem Objektcode kein Leitbefehl vorangestellt: Die Kennzeichenzeile `210000<zahl>` entfällt (im Objektcode und im Listing); der Operand von `start` hat dann keine Wirkung auf Objektcode und Listing (Abschnitt 3.3.12).

3.3.8 Definition Relativadresse

```
[<symbol>]  equ  <werteausdruck> | *
```

- Definiert für das `<symbol>` einen `<werteausdruck>`. Der Befehlszähler wird auf diesen Wert gesetzt.
- Ist `*` angegeben, wird dem Symbol stattdessen der aktuelle Befehlszählerstand zugewiesen.
- Eine Maschinencodeausgabe erfolgt nicht. Im Objektcode (Eingabe mit Kennzeichen, Format des Minimalprogramms) wird der Kennzeichenbefehl für das Setzen der Relativadresse ausgegeben (KZ=20).
- Das weitere Laden erfolgt auf 1. Leitadresse + Ladeadresse + Relativadresse. Die Ladeadresse existiert dabei nur beim Laden des Objektcodes.
- Die Anweisungen `equ100`, `equ40` und `equ10` runden den Befehlszähler (Operand `*`) oder den numerischen Operanden auf das nächste Vielfache von 0100, 040 bzw. 010 (oktal) auf.

3.3.9 Definition Absolutadresse

```
[<symbol>]  abs  <oktalzahl> | <dezimalzahl>
```

- Definiert für das `<symbol>` eine absolute `<adresse>`. Weder Objektcode- noch Maschinencodeausgabe erfolgt.
- Beachte: Symbole, die durch `abs` definiert wurden, sind nicht verschieblich. Anweisungen, in denen ein solches Symbol im Adressausdruck oder Werteausdruck vorkommt, werden mit K1K2=00 generiert!
- `abs` kann mehrfach mit dem gleichen Wert definiert werden, ohne Fehler zu erzeugen, aber nicht mehrfach mit dem gleichen Symbol für unterschiedliche Werte.

3.3.10 Definition Absolute Sektoradresse

```
[<symbol>]  spa  <oktalzahl> | <dezimalzahl>      im Bereich 00..77 (oktal)
```

- Dieser Befehl verhält sich wie ein `equ *`, nur dass die rechten beiden Oktalziffern durch den Wert von `<oktalzahl> | <dezimalzahl>`, dargestellt in 2 Oktalziffern, ersetzt werden.
- Beispiel: `spa 020` macht, wenn die Anweisung auf `*` steht, mit dem Beispielwert `*==0256` das Gleiche wie `equ 0220`. Steht `spa 020` auf 0277, wirkt sie wie `equ 0220`. Steht `spa` auf 0301, wirkt sie wie 0320.

3.3.11 Ende des verschieblichen Teils des Quelltextes

```
[<symbol>]  end
```

- Hinter dieser Anweisung sollten nur noch `abs`- oder `equ`-Anweisungen folgen. Sie werden auch dann ausgewertet, wenn das Programm Literale enthält. Makrodefinitionen (siehe Abschnitt 4.1) dürfen ebenfalls dort stehen. Der Assembler ordnet hinter dem ersten „end" die automatisch erzeugten Literaldefinitionen ein, sodass sie noch zum verschieblichen Teil hinzukommen. Ein nachfolgendes `equ` käme also HINTER die Literale. Ein weiteres „end" ändert daran nichts.
- Eine Maschinencodeausgabe erfolgt für „end" nicht.
- Code und Daten (Maschinenbefehle, `db`, `dw` usw.) hinter „end" sind nicht vorgesehen. Der Assembler lehnt sie nicht ab, sondern übersetzt sie hinter den Literalen (soweit vorhanden) mit.

3.3.12 Pragma LEITADRESSE

```
;#; LEITADRESSE <xx>
```

- Ein Pragma ist eine Kommentarzeile, die mit `;#;` beginnt und dem Assembler eine Anweisung gibt. Es ist keine Assembleranweisung im Sinne der Abschnitte 3.3.1 bis 3.3.11, erzeugt weder Maschinencode noch ein Wort auf der Trommel und steht auf einer eigenen Zeile (Einrückung mit Leerzeichen oder Tabulator ist zulässig). Ein anderes Kommentar, das mit `;#;` beginnt, bleibt ein gewöhnliches Kommentar.

- <xx> ist genau zweistellig und liegt zwischen 00 und 09 (02, nicht 2; 10 ist ein Fehler). Danach folgt nur noch Leerraum oder ein Kommentar (;). Das Schlüsselwort LEITADRESSE wird ohne Beachtung der Groß-/Kleinschreibung erkannt und vom Wert durch Leerraum getrennt.
- Das Pragma darf höchstens einmal vorkommen und muss **vor** start stehen. Ohne Pragma gilt 01.
- Wirkung: Im Objektcode (.obj) und im Listing (STDOUT und .dmp) steht anstelle des Kennzeichens 01 (K1K2=01, bezogen auf die 1. Leitadresse, siehe 3.3.7 und 3.3.8) das Kennzeichen <xx>. Das betrifft alle Zeilen, die sonst mit 01 beginnen. Zeilen mit anderen Kennzeichen bleiben unverändert. Die Dateien .test, .mqo, .sym und .log enthalten kein Kennzeichen und ändern sich nicht.
- Sonderfall ;#; LEITADRESSE 00: Der Code wird so als Objektcode (.obj) ausgegeben, wie ihn der Assembler erzeugt hat, aber mit dem Kennzeichen 00 statt 01. Das dient dazu, bereits absolut codierte Programme im Objektcode-Format auszugeben (K1K2=00: keine Adressänderung beim Laden, vgl. 3.3.9). Dem Objektcode wird dabei kein Leitbefehl vorangestellt: Die Kennzeichenzeile 210000<zahl>, die start mit Operand sonst erzeugt (3.3.7), wird weder in die .obj noch in das Listing (STDOUT und .dmp) geschrieben. Die Kennzeichenzeilen 200000<adresse> für equ bleiben erhalten. Im Objektcode sind die früheren 01-Wörter dann von den übrigen 00-Wörtern nicht mehr zu unterscheiden.
- Hat der Quelltext das Pragma, gibt der Assembler auf STDERR die Statuszeile Leitadresse: <xx> aus (nach Benutze für -s die Datei:). Das gilt auch für ;#; LEITADRESSE 01.
- Fehler (jeweils Fehler: Zeile <n> der Quelldatei: …, danach Fehler: Abbruch., Rückgabecode 16, keine Ausgabedateien):
  - Das Pragma steht hinter start: „Das Pragma LEITADRESSE muss vor start stehen.“
  - Das Pragma kommt ein zweites Mal vor: „Das Pragma LEITADRESSE darf nur einmal vorkommen.“
  - Der Wert fehlt, ist nicht zweistellig oder liegt außerhalb von 00 … 09: „Das Pragma LEITADRESSE erwartet einen zweistelligen Wert von 00 bis 09 (z. B. ;#; LEITADRESSE 02).“
- Achtung: Ein falsch geschriebenes Schlüsselwort (zum Beispiel LEITADRESS) oder ein Wert ohne Trennung vom Schlüsselwort (LEITADRESSE02) wird nicht gemeldet. Die Zeile gilt dann als gewöhnliches Kommentar, und das Kennzeichen bleibt 01. Ob das Pragma gewirkt hat, zeigt die Statuszeile Leitadresse:.

Beispiel (Projekt_1/bsp_5.16/bsp_5_16.asm, Quelltext beginnt mit der Zeile ;#; LEITADRESSE 03):

Quelltextzeile	.obj **ohne Pragma**	.obj **mit** ;#; LEITADRESSE 03
000 go	012000000.	032000000.
equ 02000	2000002000.	2000002000.
aqv: da qvektor	012027000.	032027000.
dqv: dw 05000	0000000005000.	0000000005000.
i: db 1	202007.	202007.
164 0001	010001164.	030001164.

Im Listing (.dmp und STDOUT) steht in denselben Zeilen entsprechend 03 statt 01 in der Kennzeichenspalte, zum Beispiel 0000: 03 2000 000.

Beispiel für 00 (kleines Programm, das nicht zum Projekt gehört; der Quelltext beginnt mit der Zeile ;#; LEITADRESSE 00, danach folgt start 0100):

Quelltextzeile	.obj **ohne Pragma**	.obj **mit** ;#; LEITADRESSE 00
start 0100	2100000100.	keine Zeile
anf: 211 wert	010020211.	000020211.
240 ziel	010021240.	000021240.
000 anf	010000000.	000000000.
equ 020	2000000020.	2000000020.
wert: dw 5	0000000000005.	0000000000005.
ziel: da anf	010000.	000000.
020 0	010000020.	000000020.

Im Listing (`.dmp` und STDOUT) steht entsprechend `00` statt `01` (zum Beispiel `0000: 00 0020 211`), und die Zeile `21 0000 0100` fehlt.

Tabelle 2: Sonderzeichen für die dz-Anweisung

Spalte 1: String, der als Operand bei `dz` anzugeben ist. Spalte 2: 4 Oktalziffern, die die 8 Bit des 030-Ausgabecodes enthalten.

Sonderzeichen-String	Oktalcode (030-Ausgabe)
`$wrzl`	02720
`$sm`	02720
`$inkb`	01170
`$ingb`	0570
`$irrz`	03770
`$irrs`	0160
`$trans`	01770
`$bm`	03520
`$vorl`	0
`$irrb`	0760
`$zr`	0320
`$tab`	01550
`$wr`	01120
`%`	01740
`:`	0150
`+`	0400
`` ` ``	01600
`/`	01410
`,`	01530
`.`	0730
`-`	01000

Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz

Spalten: ASCII-Kleinbuchstabe, ASCII-Großbuchstabe, Oktalcode (8 Bit des 030-Ausgabecodes), ASCII-Code Großbuchstabe (dez.), ASCII-Code Kleinbuchstabe (dez.).

Klein	Groß	Oktalcode	ASCII Groß (dez)	ASCII Klein (dez)
1	<	0010	60	49
2	"	0020	34	50
3	≈	0230	0	51
4	[	0040	91	52
5	>	0250	62	53
6	'	0260	39	54
7	≋	0070	0	55
8	~	0100	126	56
9	]	0310	93	57
0	#	0200	35	48
%	*	1740	42	37
Tabulator	Tabulator	1550	9	9

Klein	Groß	Oktalcode	ASCII Groß (dez)	ASCII Klein (dez)
q	Q	1300	81	113
w	W	1660	87	119
e	E	0450	69	101
r	R	1110	82	114
t	T	1630	84	116
z	Z	1710	90	122
u	U	1440	85	117
i	I	0510	73	105
o	O	1060	79	111
p	P	1270	80	112
:	(	0150	40	58
+	=	0400	61	43
a	A	0610	65	97
s	S	1420	83	115
d	D	0640	68	100
f	F	0460	70	102
g	G	0670	71	103
h	H	0700	72	104
j	J	1210	74	106
k	K	1220	75	107
l	L	1030	76	108
` | ; | 1600 | 59 | 96				
/	)	1410	41	47
SatzMarke	WRZL	2720	10	10
y	Y	1500	89	121
x	X	1470	88	120
c	C	0430	67	99
v	V	1650	86	118
b	B	0620	66	98
n	N	1050	78	110
m	M	1240	77	109
,	?	1530	63	44
.	!	0730	33	46
-	—	1000	—	45
Leertaste	ZwR	0320	32	32

4. Der Makro-Präprozessor des Assemblers

Der Präprozessor ist sehr einfach gehalten und hat eine starre Syntax. Er prüft, ob eine Operation ein Makroname ist. Die Prüfung erfolgt in der Textdatei `c82ass.makros`, die erweitert werden darf. Der Assembler sucht sie unter diesem festen Namen zuerst im Arbeitsverzeichnis des Aufrufs, sonst unter `Sourcecode/c82ass.makros`; fehlt sie an beiden Orten, bricht er mit einer Fehlermeldung ab. Zusätzlich können Makros direkt im Quelltext definiert werden (Abschnitt 4.1).

- Jedes dort definierte Makro beginnt mit dem Makronamen, unmittelbar gefolgt von einem Doppelpunkt und danach (getrennt durch Tab oder Leerzeichen) dem Schlüsselwort `MACRO`.

- Müssen dem Makro Parameter übergeben werden, folgt nach `MACRO`, getrennt durch Leerzeichen, die Anzahl der Parameter. Es sind maximal 9 Parameter zulässig.
- Danach kommen die Definitionsanweisungen des Makros. Sie können die formalen Parameter `$x` mit x=1..9 enthalten, die durch die Übergabeparameter ggf. mehrfach ersetzt werden.
- Jedes Makro wird durch das Schlüsselwort `MEND` abgeschlossen (Leerraum davor ist beliebig).

Im Definitionsteil kann für jeden Parameter eine Prüfliste angegeben werden. Ist sie angegeben, darf der betreffende Parameter nur die Werte der Liste annehmen. Eine Prüfliste wird durch `$PRx` mit x=1..9 definiert; sie muss auf einer Zeile für sich stehen.

Statt einer Werteliste kann hinter `$PRx` eine Typangabe stehen (z. B. `$PR1: $TYPADR`). Dann prüft der Assembler beim Makroaufruf, ob der übergebene Wert zum Typ passt; sonst meldet er einen Fehler `Parameter $x: Wert "<wert>" …`:

Typ	Zulässiger Wert beim Aufruf
`$TYPADR`	eine symbolische Adresse (ein Label, z. B. `marke`) oder eine Sternadresse (`*`, `*+3`); keine Zahl
`$TYPKON`	eine Oktalkonstante: nur Ziffern 0–7, beginnt mit `0` (z. B. `0100`); `100` oder `09` sind Fehler
`$TYPABS`	ein Symbol, das im Quelltext mit `abs` definiert ist (z. B. `absx: abs 05`)

Die Prüfung erfolgt vor der eigentlichen Übersetzung und ist deshalb nur syntaktisch: Bei `$TYPADR` wird nicht geprüft, ob das Label im Programm vorkommt (das meldet die spätere Symbolauflösung).

Die Definition der zu generierenden Anweisungen kann durch eine `$IF`-`$ELIF`-`$ENDIF`-Struktur gesteuert werden. Sie kann mehrfach auftreten, darf aber nicht geschachtelt werden. Die Schlüsselworte `$IF`, `$ELIF`, `$ENDIF` müssen jeweils auf einer Zeile für sich stehen.

Hinter `$IF` und `$ELIF` muss mindestens einer, können bis zu 4 Bedingungsblöcke angegeben werden. Ein Bedingungsblock besteht aus 3 Strings: `<wert1> <vergleichszeichen> <wert2>`. Als `<vergleichszeichen>` sind `>`, `<`, `=`, `>=`, `<=` zugelassen. Jeder folgende Bedingungsblock ist durch `|` (logisch ODER) oder `&` (logisch UND) zu verknüpfen.

Achtung: Die Bedingungsblöcke haben eine implizite Klammerung der Art:

```
(((  <bedingungsblock1> [|&] <bedingungsblock2> ) [|&] <bedingungsblock3> ) [|&] <bedingungsblock4> )
```

Innerhalb eines Definitionsblockes sind Kommentare zugelassen: `#` leitet einen Kommentar ein, der nur in der Makrodefinition gilt und auf einer eigenen Zeile stehen muss; `;` leitet einen Kommentar ein, der auch generiert wird — er kann auf einer eigenen Zeile oder in der Anweisung vorkommen.

Beispiel:

```
ANWAHL:  MACRO 1
  $PR1 4000 4100 4200 4300 4400 4500 4600 4700 K0 K1 K2 K3 K4 K5 K6 K7 LSL1 LSL2 LKL1 LKS1 LSS1 SW1I SW1O SW2O KONI
  # Generieren einer Anwahl für Zeichenkanäle
      $IF  $1  =  4000 | $1 = K7
         030 04000  ; 4000 oder K7
      $ELIF  $1  =  4100 | $1 = K6
         030 04100  ; 4100 oder K6
      $ELIF  $1  =  4200 | $1 = K5
         030 04200  ; 4200 oder K5
      $ELIF  $1  =  4300 | $1 = K4 | $1 = LKS1
         030 04300  ; 4300 oder K4 oder LKS1
      $ELIF  $1  =  4400 | $1 = K3 | $1 = LKL1 | $1 = SW2O
         030 04400  ; 4400 oder K3 oder LKL1 oder SW2 Output
      $ELIF  $1  =  4500 | $1 = K2 | $1 = LSL2 | $1 = SW1O
         030 04500  ; 4500 oder K2 oder LSL2 oder SW1 Output
      $ELIF  $1  =  4600 | $1 = K1 | $1 = LSL1 | $1 = LSS1
         030 04600  ; 4600 oder K1 oder LSL1 oder LSS1
      $ELIF  $1  =  4700 | $1 = K0 | $1 = SW1I | $1 = KONI
         030 04700  ; 4700 oder K0 oder SW1 Input oder KON Input
      $ENDIF
         MEND
```

Aufruf: `ANWAHL LSS1` erzeugt: `030  04600  ; 4600 oder K1 oder LSL1 oder LSS1`

Der Makroaufruf im Assemblerprogramm darf ein Label haben, z. B.:

```
input_loop:  ANWAHL  LSL1
             …
```

In einem solchen Fall wird generiert:

```
input_loop:  030  04600  ; 4600 oder K1 oder LSL1 oder LSS1
```

Der Makroaufruf selbst wird nicht als Kommentarzeile (`;  ANWAHL  LSL1`) ausgegeben; er erscheint weder im Listing noch in `.mqo`, `.dmp` oder `.test`.

Enthält das Makro selbst eine Schleife mit einem Label, z. B.:

```
BSP1:  MACRO
# Makro enthält eine Schleife
ML0001%: 211  …
         …
         000  ML0001%
         MEND
```

so wird bei einem Makroaufruf `loop: BSP1` folgende Anweisungsfolge generiert, wenn die erste zu generierende Anweisung ein eigenes Label hat:

```
loop      equ  *
ML0001DKR 211  …
          …
          000  ML0001DKR
```

Hat die erste Anweisung kein eigenes Label, entfällt die `equ`-Anweisung und die erste Anweisung erhält das Label des Makroaufrufs. Ausnahme: Ist die erste Anweisung `equ`, `abs`, `start` oder `end`, bleibt die `equ`-Anweisung bestehen.

Ein Makro SOLLTE kein „normales" Label der Assemblersprache enthalten, sonst kann es nur einmalig aufgerufen werden. Stattdessen muss ein generisches Label verwendet werden: genau `ML`, vier Ziffern und `%`, danach der Doppelpunkt (z. B. `ML0004%:`). Jedes andere Label mit `%` meldet den Fehler `Macrolabel muß mit ML beginnen …`. Bei jedem Aufruf wird das `%` durch drei Großbuchstaben ersetzt (z. B. `ML0001DKR`), sodass jeder Aufruf ein eigenes Label erhält.

Makro-Aufrufparameter, die exakt 4 Ziffern enthalten und nur diese, werden ggf. als 4-Ziffern-String geprüft (durch `$PRx`); sie erhalten bei der Benutzung (Ersetzung des formalen Parameters) automatisch eine Vornull. Beispiel: `ANWAHL 4600` erzeugt:

```
     030    04600  ; 4600 oder K1 oder LSL1 oder LSS1
```

4.1 Makros im Quelltext

Makros können nicht nur in der Makrodatei `c82ass.makros`, sondern auch direkt im Quelltext (`.asm`) definiert werden. Die Definition hat dieselbe Syntax wie in der Makrodatei: Sie beginnt mit dem Makronamen, unmittelbar gefolgt von einem Doppelpunkt, danach folgt das Schlüsselwort `MACRO` (bei Bedarf mit der Anzahl der Parameter). Abgeschlossen wird sie durch `MEND` (zulässig sind auch `MACROENDE` und `macroende`). Der Doppelpunkt hinter dem Makronamen ist im Quelltext Pflicht.

- **Gültigkeit:** Eine Definition darf an beliebiger Stelle im Quelltext stehen, auch vor `start`. Das Makro gilt im gesamten Quelltext, auch für Aufrufe, die vor der Definition stehen. Die Definition selbst erzeugt keinen Code.
- **Definitionsteil:** Es gelten alle Regeln dieses Abschnitts (formale Parameter `$x`, Prüflisten `$PRx`, `$IF-$ELIF-$ENDIF`, Kommentare mit `#` und `;`, generische Labels mit `%`). Ein Makro darf andere Makros aufrufen, gleichgültig ob sie in der Makrodatei oder im Quelltext definiert sind, aber nicht sich selbst.
- **Gleicher Name wie in der Makrodatei:** Das Makro im Quelltext ersetzt das gleichnamige Makro der Makrodatei, jedoch nur für diesen Assemblerlauf; die Makrodatei selbst bleibt unverändert. Der Assembler gibt dazu eine Warnung aus. Die Ersetzung wirkt auch dort, wo ein anderes Makro das ersetzte Makro aufruft.
- **Fehler:** Ein Makroname darf im Quelltext nur einmal definiert werden. Weitere Fehler sind eine `MACRO`-Zeile ohne Namen, eine Definition innerhalb einer anderen Definition (Schachtelung ist nicht erlaubt), ein fehlendes `MEND` und ein `MEND` ohne zugehöriges `MACRO`. Der Assembler meldet den Fehler und bricht nach Pass 0 ab.

Beispiel:

```
ZEILE2:  MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Aufruf `ZEILE2` erzeugt die beiden Anweisungen `dz $wrzl`.

Soll das Makro `WRZL` der Makrodatei nur für ein Programm anders arbeiten, wird es im Quelltext neu definiert:

```
WRZL:    MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Assembler meldet dazu:

```
Warnung: Makro WRZL aus der Makrodatei wird für diesen Lauf durch die Definition im Quelltext ersetzt.
```

5. Compilierungsbeispiel

```
c82ass.sh -p Projekt_1/bsp_5.16 -s bsp_5_16
```

5.1 Quelltext bsp_5_16.asm und Assemblerlauf

Hinweis: Die Datei `Projekt_1/bsp_5.16/bsp_5_16.asm` ist eine veränderte Fassung des hier abgedruckten Beispiels: Sie beginnt mit dem Startsprung `000 go` mit `equ 02000` und der Marke `go:`, enthält vor dem Halt den Debug-Befehl `164 0001`, und der Datenbereich beginnt mit `equ 02100` statt `equ 0100`. Ihr Objektcode weicht deshalb von 5.2 ab; der abgedruckte Quelltext ergibt genau den Objektcode in 5.2.

```
Projektverzeichnis: Projekt_1/bsp_5.16
Benutze für -s die Datei: bsp_5_16
Startadresse: 0000
C8205 Assembler C82aas Version 1.00-038

Pass 0
Verarbeite 'c82ass.makros', Erzeuge Macroersetzungen

Pass 1
Löse Literale auf

Pass 2
Löse Symbole auf nach Projekt_1/bsp_5.16/bsp_5_16.log und Projekt_1/bsp_5.16/bsp_5_16.sym

Pass 3
Erzeuge Kennzeichen-Befehle in Objektdatei Projekt_1/bsp_5.16/bsp_5_16.obj
```

Quelltext (`; C8205 Heft 2 Programmierung in Maschinensprache; Beispiel: 5.16 Umspeichern eines Vektors`):

```
                start
                000     beginn
aqv:            da      qvektor    ; Adresse der ersten Komponente Quellvektor
dqv:            dw      05000      ; Dimension von qvektor gezählt in Adresseinheiten
azv:            da      zvektor    ; Adresse der ersten Komponente Zielvektor
adreinheit:     dw      01000      ; Adresseinheit
spbef:          dw      0240       ; Speicherbefehl
i:              db      1          ; Arbeitszelle für i
beginn:         241     i          ; I:=0
schleife:       210     azv        ; ac:=(azv+i)
                210     spbef      ; ac:=(azv+i)240
                240     genbef     ; Arbeitszelle genbef:= (qzv+i) 240
                211     i          ; ac:=<i>
                210     aqv        ; ac:=qvektor+i
; 251: Laden des Inhaltes einer Speicherstelle, deren Adresse im AC steht, in den AC.
                251     0          ; ac:=<qvektor+i>
genbef:         db      1          ; hier steht der Befehl 240 (qzv+i) Speichern
                211     i          ; ac:=<i>
                210     adreinheit ; ac:=i+1
                240     i          ; i:=ac
                330     dqv        ; vergleicher 3, falls i == <dqv>
                170     halt       ; sprung bei gleich
                000     schleife
halt:           020     0          ; halt
qvektor:        dw      1
                dw      2
                dw      3
                dw      4
                dw      5
                equ     0100
zvektor:        db      5
ende:           end
```

Nach dem Listing meldet der Assembler:

```
Ausgabe als binaer codierte Objektdatei (R300-Lochstreifen) nach: Projekt_1/bsp_5.16/bsp_5_16.obj
Ausgabe der formatierten Speicherbelegung nach: Projekt_1/bsp_5.16/bsp_5_16.dmp
Ausgabe der Testdatei (Spalten 1,3,4,5 fuer K1=00/01) nach: Projekt_1/bsp_5.16/bsp_5_16.test
```

5.2 Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für die Eingabe mit Kennzeichen (Format des Minimalprogramms)

Das Beispiel zeigt das Listing mit Adressspalte und Leerzeichen zur besseren Lesbarkeit; die Datei `.obj` enthält die Zeilen ohne Adressspalte und ohne Leerzeichen (zum Beispiel `010007000.`), jedes Zeichen als R300-Lochbild aus acht Ziffern 0/1.

```
0000: 01 0007 000.
0001: 01 26000.
0002: 00 00000005000.
0003: 01 100000.
0004: 00 00000001000.
0005: 00 00000000240.
0006: 20 0007.
0007: 01 0006 241.
0010: 01 0003 210.
0011: 01 0005 210.
0012: 01 0016 240.
0013: 01 0006 211.
0014: 01 0001 210.
0015: 00 0000 251.
0016: 20 0017.
0017: 01 0006 211.
0020: 01 0004 210.
0021: 01 0006 240.
0022: 01 0002 330.
0023: 01 0025 170.
0024: 01 0010 000.
0025: 01 0000 020.
0026: 00 00000000001.
0027: 00 00000000002.
0030: 00 00000000003.
0031: 00 00000000004.
0032: 00 00000000005.
      20 0000 0100.
0100: 20 0105.
```